

課題 No.12 の解答例

myshell にリダイレクト機能を追加しなさい。

```
1 //
2 // myshell.c : 簡易UNIXシェル (リダイレクト機能完成版)
3 //
4 #include <stdio.h> // perror() のため
5 #include <stdlib.h> // exit(), setenv(),
6 // unsetenv() のため
7 #include <string.h> // strcmp(), strchr() のため
8 #include <unistd.h> // fork(), exec(), chdir(),
9 // close()のため
10 #include <sys/wait.h> // wait() のため
11 #include <ctype.h> // isspace() のため
12 #include <fcntl.h> // open() のため
13 #define MAXLINE 1000 // コマンド行の最大文字数
14 #define MAXARGS 60 // コマンド行文字列の最大数
15
16 int parse(char *p, char *args[]) { // コマンド行を解析する
17     int i=0; // 解析後文字列の数
18     for (;;) {
19         while (isspace(*p)) *p++ = '\0'; // 空白を '\0' に書換える
20         if (*p=='\0' || i>=MAXARGS) break; // コマンド行の終端に到達で終了
21         args[i++] = p; // 文字列を文字列配列に記録
22         while (*p!='\0' && !isspace(*p)) p++; // 文字列の最後まで進む
23     }
24     args[i] = NULL; // 文字列配列の終端マーク
25     return *p=='\0'; // 解析完了なら 1 を返す
26 }
27
28 void cdCom(char *args[]) { // cd コマンドを実行する
29     if (args[1]==NULL || args[2]!=NULL) { // 引数を確認して
30         fprintf(stderr, "Usage: _cd_ DIR\n"); // 過不足ありなら使い方表示
31     } else if (chdir(args[1])<0) { // 親プロセスが chdir する
32         perror(args[1]); // chdirに失敗したら perror
33     }
34 }
35
36 void setenvCom(char *args[]) { // setenv コマンドを実行する
37     if (args[1]==NULL || args[2]==NULL || args[3]!=NULL) { // 引数を確認して
38         fprintf(stderr, "Usage: _setenv_ NAME VAL\n"); // 過不足ありなら使い方表示
39     } else if (setenv(args[1], args[2], 1)<0) { // 親プロセスが setenv する
40         perror(args[1]); // setenvに失敗したら perror
41     }
42 }
43
44 void unsetenvCom(char *args[]) { // unsetenv コマンドを実行する
45     if (args[1]==NULL || args[2]!=NULL) { // 引数を確認して
46         fprintf(stderr, "Usage: _unsetenv_ NAME\n"); // 過不足ありなら使い方表示
47     } else if (unsetenv(args[1])<0) { // 親プロセスが unsetenv する
48         perror(args[1]); // unsetenvに失敗なら perror
49     }
50 }
```

```
51
52 char *ofile; // 出力リダイレクトファイル名
53 char *ifile; // 入力リダイレクトファイル名
54
55 void findRedirect(char *args[]) { // リダイレクトの指示を探す
56     int i, j;
57     ofile = ifile = NULL;
58     for (i=j=0; args[i]!=NULL; i++) { // コマンド行の全文字列について
59         if (strcmp(args[i], "<")==0) { // 入力リダイレクト発見
60             ifile = args[++i]; // ファイル名を記録する
61             if (ifile==NULL) break; // ファイル名が無かった
62         } else if (strcmp(args[i], ">")==0) { // 出力リダイレクト発見
63             ofile = args[++i]; // ファイル名を記録する
64             if (ofile==NULL) break; // ファイル名が無かった
65         } else { // どちらでもない
66             args[j++] = args[i]; // 文字列を args に記録する
67         }
68     }
69     args[j] = NULL;
70 }
71
72 void redirect(int fd, char *path, int flag) { // リダイレクト処理をする
73     close(fd); // リダイレクトする fd をクローズ
74     int nfd = open(path, flag, 0644); // 新しくオープン
75     if (nfd<0) { // open がエラーなら
76         perror(path); // エラーメッセージを表示
77         exit(1); // 子プロセスを終了する
78     } // プログラムにバグが無ければ他のエラーの心配は無いはず
79 }
80
81 void externalCom(char *args[]) { // 外部コマンドを実行する
82     int pid, status;
83     if ((pid = fork()) < 0) { // 新しいプロセスを作る
84         perror("fork"); // fork 失敗
85         exit(1); // 非常事態, 親を終了
86     }
87     if (pid==0) { // 子プロセスなら
88         if (ifile!=NULL) { // 入力リダイレクトがあれば
89             redirect(0, ifile, O_RDONLY); // リダイレクトする
90         }
91         if (ofile!=NULL) { // 出力リダイレクトがあれば
92             redirect(1, ofile, O_WRONLY|O_TRUNC|O_CREAT); // リダイレクトする
93         }
94         execvp(args[0], args); // コマンドを実行
95         perror(args[0]);
96         exit(1);
97     } else { // 親プロセスなら
98         while (wait(&status) != pid) // 子の終了を待つ
99             ;
100     }
101 }
102
103 void execute(char *args[]) { // コマンドを実行する
```

```
104     if (strcmp(args[0], "cd")==0) {                // cd コマンド
105         cdCom(args);
106     } else if (strcmp(args[0], "setenv")==0) {      // setenv コマンド
107         setenvCom(args);
108     } else if (strcmp(args[0], "unsetenv")==0) {   // unsetenv コマンド
109         unsetenvCom(args);
110     } else {                                       // その他は外部コマンド
111         externalCom(args);
112     }
113 }
114
115 int main() {
116     char buf[MAXLINE+2];                          // コマンド行を格納する配列
117     char *args[MAXARGS+1];                        // 解析結果を格納する文字列配列
118     for (;;) {
119         printf("Command: ");                       // プロンプトを表示する
120         if (fgets(buf, MAXLINE+2, stdin) == NULL) { // コマンド行を入力する
121             printf("\n");                          // EOF なら
122             break;                                  // 正常終了する
123         }
124         if (strchr(buf, '\n') == NULL) {            // '\n' がバッファにない場合は
125             fprintf(stderr, "行が長すぎる\n");      // コマンド行が長すぎたので
126             return 1;                               // 異常終了する
127         }
128         if (!parse(buf, args)) {                   // コマンド行を解析する
129             fprintf(stderr, "引数が多すぎる\n");    // 文字列が多すぎる場合は
130             continue;                              // ループの先頭に戻る
131         }
132         findRedirect(args);                         // リダイレクトの指示を見つける
133         if (args[0] != NULL) execute(args);        // コマンドを実行する
134     }
```

```
}

/* 実行結果
% make                                ; コンパイルエラーなし
cc -D_GNU_SOURCE -Wall -std=c99 -o myshell myshell.c
% ./myshell                            ; myshellを起動
Command: ls
Makefile      README.pdf      myshell.c
README.md     myshell
Command: echo aaa > a.txt            ; ファイルの作成をテスト
Command: cat a.txt                    ;   内容を確認
aaa                                       ;   ファイルができている
Command: ls > a.txt                    ; ファイルの上書きをテスト
Command: cat < a.txt                  ;   内容を確認
Makefile                                             ;   上書きされている
README.md
README.pdf
a.txt
myshell
myshell.c
Command: grep .pdf < a.txt            ; 入力リダイレクトをテスト
README.pdf                               ;   正しい出力がされた
Command: grep .pdf < a.txt > b.txt     ; 入出力の同時リダイレクト
Command: cat b.txt                     ;   正しく処理されている
README.pdf
Command: grep .pdf < c.txt            ; 入力リダイレクトのエラー処理
c.txt: No such file or directory       ;   正しいエラーメッセージ
Command: grep .pdf > /d.txt           ; 出力リダイレクトのエラー処理
/d.txt: Read-only file system          ;   正しいエラーメッセージ
Command: grep .pdf < c.txt > /d.txt   ; 同時リダイレクトのエラー処理
c.txt: No such file or directory       ;   片方のメッセージだけ表示
Command:
*/
```